

Transformer Architecture in Large Language Models

Document Summary

This note covers the foundational elements of the Transformer architecture, including tokenization methods like BPE, positional encodings, the attention mechanism with self-attention and cross-attention variants, and multi-head attention. It explores encoder-decoder structures, encoder-only models like BERT, decoder-only models like GPT, and generation techniques such as sampling methods. Key advantages over RNNs, such as parallelization and handling long-range dependencies, are highlighted for large language models.

Introduction to Transformers

The **transformer** (Vaswani et al., 2017) is a **seq2seq** architecture proposed as **encoder-decoder**. **Encoder-only** and **decoder-only** variants exist. Unlike RNN-based models, transformers handle long-term dependencies without $O(N^2)$ complexity for long sequences. They avoid unrolling N times, preventing gradient vanishing/explosion, and enable parallelization. #computer-science/machine-learning #level/intermediate

To clarify, this means transformers process the entire sequence at once rather than step-by-step, which allows for efficient training on modern hardware. Key aspects covered include:

- Tokenization
- Positional encoding
- **Attention mechanism**
- (Multi-head) Self-attention & cross-attention

Reference: "Attention is all you need", <https://arxiv.org/pdf/1706.03762>

Encoder-Decoder Overview

The **encoder-decoder** structure forms the foundation of the transformer. Let's break it down step by step.

- **Encoder:** Processes the entire input sequence in parallel, producing a contextualized representation (one code vector per input token).
- **Decoder:** Generates the output sequence autoregressively, starting from a BOS (Beginning of Sequence) token. It predicts one token at a time while conditioning on previous outputs-

Outputs: Probability distributions over the vocabulary for each position; extract the next token via argmax (selecting the highest probability) or sampling (random selection based on probabilities).

- Inference: Uses the latest predicted token generate the next one. Training: Computes the loss across all tokens in the target sequence using teacher forcing (providing the ground-truth previous tokens).

Generating the 1st Token

- *Encoder: Encodes the full input sequence once.*
- *Decoder: Feeds only the BOS token as input.*
- *Output: Probability distribution; select the first generated token (tok1).*

Generating the 2nd Token

- *Encoder: Reuses the same encoded representation (no recomputation needed).*
- *Decoder: Feeds BOS + tok1 as input.*
- *This process continues iteratively until an EOS (End of Sequence) token is generated or a maximum length is reached.*

For simplicity in the initial discussion (details revisited later), we make these assumptions:

1. Ignore residual connections.
2. Assume inputs and outputs are raw token vectors.
3. Ignore Add & Norm layers.

The core building blocks are attention blocks (which take 3 inputs—queries, keys, values—and produce 1 output) and feed-forward (FF) networks.

General Architecture

A transformer consists of a stack of encoder layers followed by a stack decoder layers (e.g., N=6 layers each in the original model). This stacked design allows the model to build increasingly abstract representations. #note/process

- **Encoder Stack:** Each layer includes Multi-Head Attention (self-att on the input) followed by a Feed-Forward network, with residual connections and layer normalization (Add & Norm) after each sub-block.

- **Decoder Stack:** Each layer includes Masked Multi-Head Attention (self-attention on the output, masked to prevent future peeking), Multi-Head Attention (cross-attention from the encoder's output), and a Feed-Forward network, again with Add & Norm after each.
- Input: Token embeddings combined with Positional Encoding to inject sequence order.
- Output: A linear projection layer followed by softmax to produce token probabilities over the vocabulary.

$N \times$ denotes the N stacked layers in each stack.

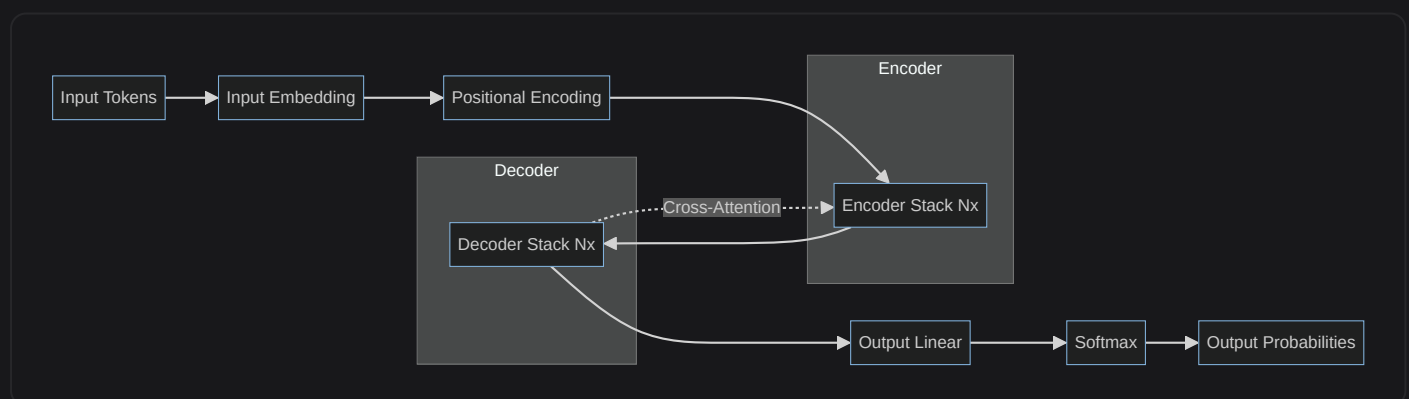
Simplified Architecture Diagram Description

To visualize the flow, consider the following high-level structure:

- Inputs (token) → Input Embedding → Positional Encoding → Encoder Stack ($N \times$ layers: Multi-Head Attention → Add & Norm → FF → Add & Norm).
- Encoder output (contextualized representations) → Decoder Stack ($N \times$: Masked Multi-Head Attention → Add & Norm → Multi-Head Cross-Attention (using encoder output) → Add & Norm → FF → Add & Norm).
- Decoder output → Output Linear Projection → Softmax Output Probabilities (over vocabulary).

During training, the decoder inputs are shifted right (starting with BOS) to align predictions with targets. This setup ensures the model learns to predict the next token given the previous ones.

For a clearer illustration of the component relationships, here's a Mermaid diagram showing the system architecture:



Tokenization

Transformers process sentences by first splitting them into **tokens** through a process called **tokenization**. This step converts raw text a sequence of discrete units that the model can handle numerically. #note/definition

Previous tokenization options include:

- Character-level (prone to issues like case sensitivity in corrections).
- Word-level (using pre-trained embeddings like word2vec).
- Subword-level (as in fastText with n-grams).

Reference: Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with word units." arXiv preprint arXiv:1508.07909 (2015).

Subword-level tokenization is the most common approach today, except in rare cases like CharBERT.

Reference: Ma,ao, Yiming Cui, Chenglei Si, Ting Liu, Shijin Wang, and Guoping Hu. "CharBERT: Character-aware pre-trained language model." arXiv preprint arXiv:2011.01513 (2020).

Popular subword variants include Byte-Pair Encoding (BPE), WordPiece, SentencePiece, and Morfessor.

Character-Level Tokenization

Character-Level Tokenization > **Description:** Breaks the text into individual characters, treating each as a separate token. This results in a very fine-grained representation.

Pros:

- No Out-Of-Vocabulary (OOV) issues, every character is known.
- Robust to misspellings, typos, or morphological variations.

Cons:

- Produces much longer sequences, exacerbating the $O(N^2)$ complexity in transformers.
- to slower and more computationally intensive training.
- Harder for the model to capture higher-level semantics like word meanings.
- Inefficient for common words, which become excessively fragmented.

Character-Level Tokenization

- *Input:* Mrs. Rucastle was downstairs, so I had an admirable opportunity.
- *Tokens:* M r s . R u c a s t l e w a s d o w n s t a i r s , s o I h a d a n a d m i r a b l e o p p o r t u n i t y .

Word Tokenization

Word-Level Tokenization

Description: Splits the text into whole words, with each word serving as a token. Punctuation and spaces are handled separately.

****:

- Directly captures semantic meaning at the word level.
- Results in shorter sequences compared to character-level.
- More intuitive and easier for humans to interpret.

Cons:

- OOV problems for rare, new or unseen words (e.g., proper names or neologisms).
- Does not leverage shared substructures like prefixes or suffixes across words.
- Requires a larger vocabulary size, leading to memory inefficiency.
- Rare receive less training data, degrading performance.

Word-Level Tokenization

- *Input:* Mrs. Rucastle was downstairs, so I had an admirable opportunity.
- *Tokens:* Mrs . Rastle was downstairs , so I had an admirable opportunity .

Subword-Level Tokenization

Subword-Level Tokenization

Description: Divides into subword units, such as common prefixes, suffixes, or morphemes. For instance, fastText uses character *n*-grams. The most common method is BPE, which merges frequent pairs iteratively.

Pros- Strikes a balance between character-level granularity and word-level semantics.

- Effectively handles OOV words by breaking them into known subwords.
- Uses a compact vocabulary, promoting better generalization.
- Efficient for both frequentwhole words) and rare words (composed subwords).

Cons:

- Common words are represented by fewer subwords, while rare ones use more, though this is generally beneficial.
- Requires defining a subword policy based on the training corpus.
- Introduces some computational overhead during encoding/decoding.

Subword-Level Tokenization (using BERT base tokenizer, cased)

- *Input: Mrs. Ruc was downstairs, so I had an admirable opportunity.*
- *Tokens: Mrs . R uca st le was downstairs , so I had an ad mir able opportunity .*

Note Common words are represented by fewer subwords compared to rarer ones, which helps in efficient modeling.

Byte-Pair Encoding (BPE)

Byte-Pair Encoding (BPE)

Description BPE is a data-driven algorithm that learns a compact vocabulary of subwords from a training corpus. It starts with individual characters and iteratively merges the most frequent adjacent pairs into new tokens. The goal is to represent sequences as single tokens and rare ones as compositions. Typically, the vocabulary size is set to 10k-100k tokens. It handles non-ASCII characters via UTF-8 encoding (base-256).

Iterative: The method ensures token selection is empirical, reducing sequence length while maintaining expressiveness. Frequent merges follow Zipf's law, where a few high-frequency items account for most occurrences.

Algorithm (step-step):

1. Encode the entire corpus treating each character (or byte) as an initial token.
2. Count the frequency of every pair of adjacent tokens in the corpus.
3. Identify the most frequent pair (F1, F2) and create a new token T representing their merge.
4. Replace all occurrences of (F1, F2) with T throughout the corpus.
5. Repeat steps 2-4 until desired vocabulary size is reached.

To illustrate with a simple Python snippet for the merging step (conceptual; full implementation requires a library like Hugging Face's tokenizers):

```
# Example Python snippet for a BPE merge step
def merge_pairs(corpus_tokens, pair_freqs):
```

```

"""
Merge the most frequent pair in a list of tokens.
corpus_tokens: List of token lists (one per sentence)
pair_freqs: Dict of (token1, token2): frequency
"""

most_freq_pair = max(pair_freqs, key=pair_freqs.get)
new_token = ''.join(most_freq_pair) # Merge into single token
updated_corpus = []
for sentence in corpus_tokens:
    new_sentence = []
    i = 0
    while i < len(sentence):
        if i + 1 < len(sentence) and (sentence[i], sentence[i+1]) =
            new_sentence.append(new_token)
            i += 2
        else:
            new_sentence.append(sentence[i])
            i += 1
    updated_corpus.append(new_sentence)
return updated_corpus

# Hypothetical usage
corpus = [['w', 'o', 'u', 'l', 'd', ' ', 'a', ' ']] # Simplified
pair_freqs = {('w', 'o'): 1, ('o', 'u'): 1} # From counting
new_corpus = merge_pairs(corpus, pair_freqs)
print(new_corpus) # [['wo', 'u', 'l', 'd', ' ', 'a', ' ']]

```

BPE Example (using corpus: "would a woodchuck chuck wood")

- Step 1: Initial encoding (28 tokens, including spaces as `_`): `| w | o | u | l | d | _ | a | _ | w | o | o | d | c | h | u | c | k | _ | c | h | u | c | k | _ | w | o | o | d |`
- Initial vocabulary: `{ _, a, c, d, h, k, l, o, u, w }`
- Step 2: Count pairs (partial list for brevity>
 - `w,o`: 3
 - `o,u`: 1
 - `u,l`: 1
 - `l,d`: 1
 - `d,_`: 1
 - `_,a`: 1

- $a, _ : 1$
- $_, w : 2$
- $o, o : 2$
- $o, d : 2$
- $d, c : 1$
- $c, h : 2$
- $h, u : 2$
- $u, c : 2$
- $c, k : 2$
- $k, _ : 2$
- $_, c : 1$
- Step 3: Most frequent pair: w, o (frequency 3) → Create new token $|wo|$.
- Updated vocabulary: $\{ _, a, c, d, h, k, l, o, u, w, wo \}$
- Step 4: Replace all $|w/o|$ with $|wo|$ in the corpus:
 - Updated corpus: $|wo|u|l|d|_|a|_|wo|o|d|c|h|u|c|k|_|c|h|u|c|k|_|wo|o|d|$
- iterations (example progression for illustration):
 - Next merge: $|wo/o| \rightarrow |woo|$ (from "woo" in woodchuck/wood).
 - Then: $|woo/d| \rightarrow |wood|$.
 - Parallel merges: $|c/h| \rightarrow |ch|$, then $|ch/u| \rightarrow |chu|$, $|chu/c| \rightarrow |chuc|$, $|chuc/k| \rightarrow |chuck|$.
 - $\therefore |w/o/u| \rightarrow |wou|$ (from "would"), and so on.
 - The process continues until the target vocabulary size is achieved.
- Note: If multiple the same frequency, any can be chosen. Early merges have the most impact due to Zipf's law, reducing token count dramatically at first.

To track the reduction in token count across steps, consider this table for the corpus (hypothetical full run to 15 merges):

MERGE STEP	MOST FREQUENT PAIR MERGED	NEW TOKEN COUNT PER SENTENCE (AVG)	VOCABULARY SIZE
0 (Initial)	N/A	28	10
1	w, o	26	11

MERGE STEP	MOST FREQUENT PAIR MERGED	NEW TOKEN COUNT PER SENTENCE (AVG)	VOCABULARY SIZE
2	o,o	24	12
3	c,h	22	13
...	...	...	...
15	Various (e.g., chuck, wood)	12	25

B Results:

- Token count decreases progressively with more merges, shortening sequences and improving efficiency.
- Initial merges are most impactful, capturing high-frequency patterns.
- Shorter sequences reduce computational cost, while subwords preserve better than characters.
- This avoids overfitting to the specific corpus by generalizing subword compositions.

Special Tokens

Special tokens are reserved vocabulary entries that serve structural or task-specific roles:

#note/definition

- ******** (Beginning of Sequence): Marks the start of a generated sequence.
- **EOS** (End of Sequence): Signals the end of generation.
- **CLS** (Classification): Used in models like BERT for aggregating sequence in classification tasks (similar to BOS but task-oriented).
- **SEP** (Separator): Divides multiple segments, such as in question-answering (e.g., query [SEP] context).
- **PAD (Padding)**: Fills shorter sequences to a fixed length during batching (e.g., used as BOS in some models like T5).

Special Tokens

Example: In a translation, input might be [BOS] English sentence [SEP] [PAD]..., ensuring consistent handling.

Positional Encoding

Token embeddings map each token to a fixed learned vector, but this vector is the same regardless of token's position in the sequence. However, the attention mechanism treats the input as an unordered set, not a sequence—position is crucial for understanding order!

#computer-science/machine-learning #level/advanced

Solution: address this, add a **positional encoding (PE)** vector to each token's embedding, specific to its position. This injects sequential information into the model.

In the encoder: Input Embedding + Positional Encoding → Encoder.

Need for PE

Without positional encoding, the model cannot distinguish order. Consider this example:

No PE

- *Input 1: I ate an apple → Embeddings: $vec_I, vec_{ate}, vec_{an}, vec_{apple}$.*
- *Input 2: An apple I ate → Embeddings: $[vec_{an}, vec_{apple}, vec_I, vec_{ate}]$ (set, just reordered).*

The model would treat these as identical, losing syntactic meaning. With PE, positions differentiate them:

With PE

- *Input 1 $[vec_I + PE_1, vec_{ate} + PE_2, vec_{an} + PE_3, vec_{apple} + PE_4]$.*
- *Input 2: $[vec_{an} + PE_1, vec_{apple} + PE_2, vec_I + PE_3, vec_{ate} + PE_4]$.*

Note: This is a simplified illustration; actual transformers use sophisticated PE.

Sinusoidal Positional Encoding (AIAYN)

The original transformer uses fixed (non-learned) sinusoidal positional encodings. These are deterministic and defined for each position `pos` (starting from 0) and `i` (from 0 to $d_{model}/2 - 1$):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- d_{model} : The embedding dimension (e.g., 512 or 768 in common models).
- Even (2i) use sine; odd dimensions (2i+1) use cosine.
- The denominator creates varying frequencies: low i (early dimensions) change slowly (capturing broad position info), while high i oscillate rapidly (fine-grained details).

Sinusoidal PE (for $pos=1$, $d_{model} = 768$, first few dimensions)

- For $i=0$ (dimension 0): $\sin(1/10000^{0/768}) = \sin(1) \approx 0.8415$
- For $i=0$ (dimension 1): $\cos(1/10000^{0/768}) = \cos(1) \approx 0.5403$
- For $i=1$ (dimension 2): $\sin(1/10000^{2/768}) = \sin(1/10000^{0.0026}) \approx \sin(0.99997) \approx 0.8415$ (but scales down slightly)
- For $i=1$ (dimension 3): $\cos(1/10000^{2/768}) \approx 0.5403$

As i increases, the frequency rises, allowing the model encode position uniquely.

Uniqueness: These encodings are unique for up to approximately 60,000 positions (based on the 2π period scaled by 10,000), after which they begin to repeat. The constant (10,000) can be adjusted for longer sequences if.

Preserves Similarity: Positional encodings for nearby positions remain similar, which helps the model generalize smooth changes in position. This can be using trigonometric identities, such as the fact that rotations preserve distances.

Similarity in PE (for max $pos=1000$, $d_{model} = 768$)

If you compute the cosine similarity between vectors, the diagonal (same position) is 1.0, and it decreases gradually with positional distance, forming a clear "heat map" of similarity.

Learned Positional Embeddings

- Alternative to: Learned positional embeddings, treated as trainable parameters alongside token embeddings.
- Used in the GPT family of models (not sinusoidal).
- The original "Attention is All You Need" paper experimented with learned PE but discarded it in of sinusoidal (no performance gains observed).

- --2 specifically employs learned positional embeddings, allowing the model to adapt them during training.

Attention Mechanism

Attention is the core innovation of transformers: it the model to contextualize independent input vectors by computing a weighted sum of them. The weights are learned based on the relevance within the sequence, enabling dynamic focus. #ai/language-models #level/advanced

Attention is used in main places: encoder self-attention (tokens attending to each other), decoder masked self-attention (past tokens only), and encoder-decoder cross-attention (decoder attending to encoder outputs).

In simplified terms, an layer takes three inputs—queries (Q), keys (K), and values (V)—and produces a contextualized output for each query.

Simplified Attention

To make this concrete, think of attention as determining "much" of each input vector to incorporate into the output. The attention weights decide the mixing proportions (e.g., emphasizing relevant tokens more).

- **Output:** A weighted sum of the value vectors.
- **Weights:** Computed a softmax over similarity scores between queries and keys.

For the i -th output position:

$$out_i = \sum_j AttnWeight_{i,j} \cdot V_j$$

The attention mechanism itself computes these $AttnWeight_{i,j}$ values.

Analogy: Dictionary Lookup

Imagine a dictionary with key-value pairs. A query looks up the most matching key and retrieves its value:

- **Classic discrete:** Exact match (0 or 1 weight).
- **Continuous attention:** Use dot-product similarity (query · key) for a soft, probabilistic match.

- Result: Output = $0.2 \cdot \text{value_A} + 0.7 \cdot \text{value_C} + 0.03 \cdot \text{value_D} + 0.07 \cdot \text{value_E}$ (weights sum to 1).

This analogy highlights how attention retrieves and blends relevant information dynamically.

Keys Values, Queries

- **Queries (Q):** what the current token is "asking" for—one query per output position.
- **Keys (K):** Represent what can be matched against (one per input to attend to).
- **Values (V):** actual content to retrieve, associated with each key blended via weights).

These form a 3-input, 1-output block in the architecture diagrams.

From-Product to Weights

- **Similarity Score:** Computed as the dot product $query \cdot key^T$, which is high if vectors align (low if orthogonal, e.g., 0).
- **For one query all keys:** Produces a row vector of N similarities (for N positions).
- **Across all queries:** An $N \times M$ similarity matrix (N queries, M keys).
- **Scaling:** Divide by $\sqrt{d_k}$ (where d_k is key dimension) to prevent dot products from growing too large in high dimensions, stabilizing gradients (assumes Q and K are unit variance).
- **Softmax:** Applied row-wise to similarities into probabilities (each row sums to 1, values in $[0,1]$).

Matrix Form

- **Q shape:** $N \times d_k$ (N queries, each d_k -dimensional).
- **K shape:** $M \times d_k$ (M keys).
- **V shape:** $M \times d_v$ (M values, d_v often equals d_k).

The Scaled Dot- Attention formula is:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- QK^T : $N \times M$ matrix of raw dot products.
- **Scaled:** Keeps variance around 1 for stable softmax.
- **Softmax:** Produces $N \times M$ attention weights.
- **Multiply by V:** Yields $N \times d_v$ contextualized outputs.

Numerical Attention (for 3 tokens, $d_k = 2$, simplified vectors)

- $Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, V = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$
- $QK^T = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}$
- Scaled ($\sqrt{2} \approx 1.414$, divide by 1.414): $\begin{pmatrix} 0.707 & 0 & 0 \\ 0 & 0.707 & 0 \\ 0.707 & 0.707 & 0 \end{pmatrix}$
- Softmax (per row): Approx. $\begin{pmatrix} 1.0 & 0 & 0 \\ 0 & 1.0 & 0 \\ 0.5 & 0.5 & 0 \end{pmatrix}$
- $\times V: \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 3 & 5 \end{pmatrix}$ (first row takes V1 fully, third blends V1 and V2).

Generating Q, K, V

- Derived from the input embeddings via separate linear projections: $Q = XW^Q, K = XW^K, V = XW^V$ (X is input matrix).
- In cross-attention: Q from one source (e.g., decoder), K/V from another (e.g., encoder).

Attention Example

Starting from input token embeddings (say, 3 tokens with $d_{model} = 4$), project via W^Q, W^K, W^V (each $4 \times d_k$, assume $d_k = 2$) to get Q, K, V.

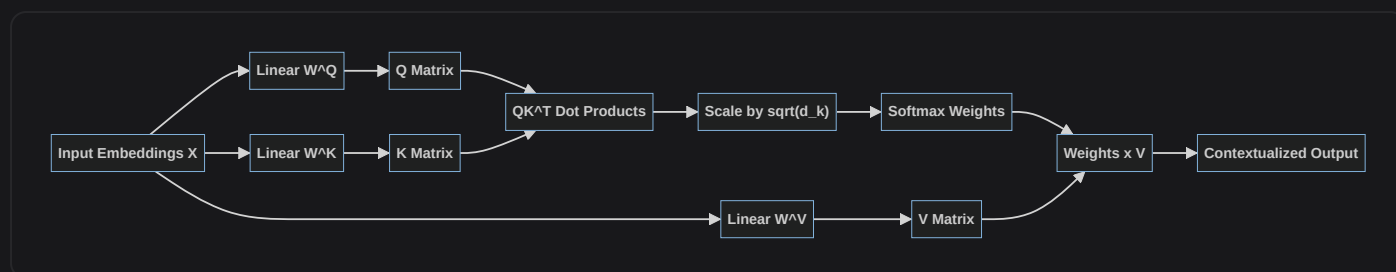
Dot-products (partial matrix for first query): [0.1, 0.5, 0.1] against keys.

Full computation (hypothetical 3x3 similarity matrix before scaling):

- Raw: $\begin{pmatrix} 0.2976 & 0.1524 & 0.936 \\ \dots & & \end{pmatrix}$
- Scaled (divide by $\sqrt{2} \approx 1.414$): $\begin{pmatrix} 0.2104 & 0.1078 & 0.6619 \\ \dots & & \end{pmatrix}$
- Softmax (first row): [0.2879, 0.2598, 0.4522]
- Weighted V (assume V rows: [0.1,0.2], [0.3,0.4], [0.5,0.6]): Output for first = $0.2879 * [0.1,0.2] + 0.2598 * [0.3,0.4] + 0.4522 * [0.5,0.6] \approx [0.15, 0.17]$ for first two dims (extend for full).

This repeats for each query row, producing contextualized vectors.

For a visual of the attention computation flow:



Types of Attention

Attention variants adapt to different parts of the architecture, ensuring appropriate information flow. #note/process

Encoder Self-Attention

Encoder Self-Attention

- *Description: Each token in the input attends to all other tokens in the same input sequence, capturing full bidirectional context.*
- *Q, K, V are all derived from the same encoder input sequence.*
- *No masking: Every position can "see" the entire sequence.*

This allows the encoder to build rich, global representations.

Decoder (Masked) Self-Attention

Decoder (Masked) Self-Attention

- *Description: Applied to the decoder's output sequence, but causally masked to ensure each token only attends to previous tokens (and itself), simulating autoregressive generation.*
- *Masking: Future positions get $-\infty$ in the attention logits (becomes 0 after softmax).*
- *For the first token: Attends only to itself.*
- *For the third token: Attends to positions 1, 2, and 3.*

The formula incorporates a mask:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{Mask}\right) V$$

Note: The mask is added before softmax. In practice, compute the full matrix and mask unnecessary parts (efficient on GPUs due to parallelization).

Masking Example

For sequence [BOS, tok1, tok2], the mask for tok2's attention is upper-triangular (zeros for future).

Encoder-Decoder Cross-Attention

Encoder-Decoder Cross-Attention

- *Description: Allows the decoder to incorporate information from the encoder, focusing on relevant input parts during generation.*
 - *Q from the decoder's current layer; K and V from the encoder's final output.*
 - *No masking: Each decoder token attends to the entire encoder sequence.*
 - *Handles varying lengths: N decoder positions query encoder positions.*
- This bridges the input and output spaces effectively.*

Multi-Head Attention

A single attention mechanism is like one "head" focusing on a specific pattern (W^Q, W^K, WV unique per head**Multi-Head Attention**: Runs h parallel attention heads (each with its own projections), allowing the model to attend to information from different representation subspaces simultaneously (e.g., syntax in one head, semantics in another).
#ai/language-models

- Compute h independent attentions.
- Concatenate the outputs: [head_1 || head_2 || ... || head_h].
- Apply a final linear projection W^O to map back to d_{model} .

The formula is:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Example: BERT uses $h=12$ heads, each with $d_k = d_v = 64$ (for $d_{model} = 768$), enabling diverse.

This multi-head design improves expressiveness without increasing depth.

Additional Details

Residual Connections

Residual Connections

- *Description: Skip connections that add the input directly to the sub-block output: $out = Layer(x) + x$.*
- *Purpose: Improves gradient flow during backpropagation, mitigating vanishing gradients and enabling training of deeper networks.*
- *Origin: Popularized in ResNet for computer vision.*

In the transformer architecture: After each Attention or sub-block $\rightarrow$ Add (residual: input + sub-output) $\rightarrow$ LayerNorm.

Residual Example

If input $x = [1, 2]$, Attention(x) = $[1.1, 2.2]$, residual = $[1.1, 4.2]$, preserving original signal.

Layer Normalization

Layer Normalization

- *Description: The "Norm" in Add & Norm layers normalizes activations across the feature dimension for each token independently.*
- *Formula a vector $[x_1, x_2, \dots, x_d]$ (per token):*

$$y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

- $\mu = E[x]$: Mean across dimensions.
- $\sigma^2 = Var[x]$: Variance across dimensions.
- ϵ : Small constant for numerical stability (e.g., $1e-6$).

- γ, β : *Learned scale and shift parameters (allow the model to adjust the normalization).*

Applied per sample (token), rescaling to a standard range (mean 0, variance 1) before scaling/shifting.

Benefits: Stabilizes training, reduces sensitivity to initialization, and speeds convergence.

Numerical Layer Norm (for vector [1, 3, 5], $\epsilon=3$)

- $\mu = (1 + 3 + 5)/3 = 3, \sigma^2 = [(1 - 3)^2 + (3 - 3)^2 + (5 - 3)^2]/3 = 8/3 \approx 2.667, \sqrt{\sigma^2 + \epsilon} \approx 1.633$
- *Normalized: $y1 = (1-3)/1.633 \approx -1.224, y2=0, y3=1.224$*
- *With $\gamma = 1, \beta = 0$: [-1.224, 0, 1.224]*

Stacking

- Both encoder and decoder consist of N identical layers stacked sequentially (e.g., N=6 in the original paper; smaller N=3 for lightweight examples).
- The output of one layer feeds as input to next, allowing hierarchical feature extraction (low layers capture local patterns, high layers global).

Relative Positional Embeddings

Relative Positional Embeddings

- *Description: An alternative to absolute positional encodings, where position information is added relative to the between query and key positions during attention computation.*
- *Not added to input embeddings; instead, incorporated into the attention scores.*
- *Formula:*

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T + R}{\sqrt{d_k}} \right) V$$

- *R: A learned relative position matrix (S_{rel}), encoding biases for different distances.*
- *Primarily for self-attention (more for cross-attention due to separate sequences).*
- *Used in modern language models as an alternative to absolute learned or sinusoidal PE; examples include rotary embeddings (RoPE).*

Advantages of Transformers

Transformers offer several key benefits over prior like RNNs, making them ideal for large language models. #note/info

- **Parallelization:** All tokens are processed simultaneously without sequential dependencies (unlike RNNs). Teacher forcing during training further enables efficient batching on GPUs/TPUs.
- **Long-Range Relationships:** The attention mechanism directly connects any two positions in the sequence, allowing focus on distant dependencies without information decay ($O(N^2)$ time/space is a trade-off for long).
- **Better Performance:** No sequential unrolling means no forgetting of early information, leading to superior handling of long contexts compared to RNNs.

In summary, these properties enable scaling to massive models and datasets.

Encoder-Decoder Architecture

The vanilla transformer uses an encoder-decoder setup for sequence-to-sequence tasks, such as machine translation, where input and output are different sequences.

T5 (Text-to-Text Transfer Transformer)

T5 (Text-to-Text Transfer Transformer)

Description: T5 reframes NLP tasks as text-to-text problems, using task-specific prefixes (e.g., "translate English to German:") to guide the model. This unified approach eliminates the need for task architectures.

- *A single pre-trained handles diverse tasks like translation, summarization, and question-answering.*
- *Input: Prefix + input sequence.*
- *Output: Generated text sequence.*

Reference: Raffel, Colin et al. "Exploring the limits transfer learning with a unified text-to-text transformer." Journal of machine learning research 21.140 (2020): 1-67.

[https://arxiv.org/pdf/191010683](https://arxiv.org/pdf/1910.10683)

T5 Examples

- *Input: translate english to german: what is your profession?*
- *Output: <pad> Was ist dein Beruf? </s>*
- *Input: translate english to german: What is your?*
- *Output: Was ist Ihr Beruf? </s>*

Here, <pad> acts as BOS; </s> as EOS.

Not Instruction-Tuned:

- T5 is tuned on task prefixes and performs well on seen formulations but generalizes poorly to new phrasings or tasks without fine-tuning.
- This highlights the need for instruction tuning in modern models.

Examples of Poor Generalization

- *Input: can translate from English to German, What is your profession? → Output: Was ist Ihr Beruf? (may work if similar, but inconsistent).*
- *Input: can you translate from English to German the following sentence What is your? → Output: <unk> <unk>... (fails on novel structure).*
- *Input: English: what is your profession? German: → Output: Deutsch: Deutsch:... (misinterprets format).*
- *Input: compute: 2+2 = → Output: 2+2+2+2+2 +2+2+2+2+... (repetitive error on math task).*

Beyond Encoder-Decoder: Encoder-Only

Encoder-only models focus solely on understanding and encoding the input sequence, without autoregressive generation. The decoder is removed, and a task-specific head (e.g., classifier) is added on of the encoder output.

- Suitable for tasks like text classification, Named Entity Recognition (NER), or sentiment analysis, where the goal is representation learning rather than generation.

BERT (Bidirectional Encoder Representations from Transformers)

BERT (Bidirectional Encoder Representations from Transformers)

- *Description: A seminal encoder-only model, pre-trained in a self-supervised manner and fine-tuned for downstream tasks. It uses full bidirectional self-attention, allowing each token to attend to all others in both directions- Pre-training objectives: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).*
- *Architecture: 12 layers (or 24 for large), 12 heads, $d_{model} = 768$.*

Reference: Devlin, Jacob. "BERT: Pre-training of deep bidirectional transformer representations for language understanding." arXiv preprint arXiv:1810.04805 (2018).

<https://arxiv.org/pdf/1810.04805>

Input Encoding:

- Handles single sentences or pairs: [CLS] Sentence A [SEP] Sentence B [SEP].
- [CLS] token's final representation is used for sequence-level tasks (e.g., classification).
- Additional segment embeddings distinguish A/B segments (binary vector added to token + positional embeddings).

Masked LM:

- Since bidirectional attention prevents simple next-token prediction, randomly mask ~15% of tokens and predict them from context.
- The model outputs predictions for all masked positions via a linear layer over the vocabulary.
- Example: Input [CLS] my [MASK] is [MASK] [SEP] → Predict "dog" for first mask, "cute" for second (from "my dog is cute").

Next Sentence Prediction:

- Objective: Predict if sentence B follows A (binary classification via [CLS] output).
- Training data: 50% true pairs, 50% random.
- Example (Yes): A: I went to the store... [SEP] B: When I got home... [SEP] → Label: Yes.
- Example (No): A: I went to the store... [SEP] B: The weather is nice. [SEP] → Label: No.

Attention in BERT:

- With 12 layers and 12 heads, for an 11-token sentence: Each head computes 11×11 attention maps, stacked across layers.
- Example sentence: "[CLS] The dog ate the food because it was hungry [SEP]"
- Token "ate" might attend strongly to "dog" (subject) and "food" (object).
- Token "it" attends to "dog" (resolves pronoun ambiguity, not "food").
- In final layers, attention may focus on [SEP] for boundary awareness (though sometimes a "no-op").

Reference: Michel, Paul, Omer Levy, and Graham Neubig. "Are sixteen heads really better than one?." Advances in neural information processing systems 32 (2019).

BERT's bidirectional nature excels at understanding tasks but cannot generate text directly.

Beyond Encoder-Decoder: Decoder-Only

Decoder-only models extend the input sequence autoregressively for generation, without a separate encoder. The input prompt serves as the starting point for the decoder stack.

- No encoder-decoder cross-attention (all self-attention, masked).
- Possible extensions: Cross-attention to other modalities (e.g., images in multimodal models).
- Example: The GPT family of models.

GPT (Generative Pre-trained Transformer)

GPT (Generative Pre-trained Transformer)

- *Description: A decoder-only architecture pre-trained on next-token prediction: $P(w_t | w_1, \dots, w_{t-1})$, modeling language as a causal chain.*
- *After pre-training, fine-tune on supervised tasks.*
- *Input: The prompt sequence (starts generation).*
- *Generation: Autoregressively sample or select the next token based on the decoder output.*

Reference: Radford, Alec et al. "Improving language understanding by generative pre-training." (2018). <https://hayate-lab.com/wp-content/uploads/2023/05/43372bfa750340059ad87ac8e538c53b.pdf>

GPT-2 Generation

- *Prompt: "I had called upon my friend, Mr. Sherlock Holmes..."*
- *Generated continuation: "...with the full intention of telling him my troubles." (coherent, contextually appropriate).*

This setup enables open-ended text generation from prompts.

Sampling Approaches

The transformer's output at each step is a probability distribution over the vocabulary (e.g., 50,257 tokens for GPT-2). Selecting the next token involves a policy to balance determinism, creativity, and coherence. #note/process

Next Token Selection Policies:

- **Deterministic:** Always reproducible, often leading to safe but bland outputs.
- **Stochastic:** Introduces variability for more creative or diverse generations.

Greedy Sampling

Greedy Sampling

- *Description:* At each step, select the token with the highest probability (argmax).
- *Pros:* Deterministic and efficient.
- *Cons:* Prone to repetition, as it gets stuck in local optima.

Greedy (GPT-2 on a prompt)

Output: "I am afraid I have not seen you in a long time. I am afraid I have not seen you in a long time..." (endless loop).

Beam Search

Beam Search

- *Description:* Maintains a beam of the top-k most probable partial sequences. At each step, expand each by possible tokens, score the new sequences (e.g., joint probability), and prune to keep only the top-k.
- *Pros:* Explores multiple paths before committing, reducing errors; deterministic.
- *Cons:* Can still be repetitive; computationally heavier.

Beam Search Illustration (beam width k=3, prompt "The woman")

- *Step 1:* Top candidates: "has a dog" (prob 0.4), "went to the house" (0.3), "is walking" (0.2).
- *Step 2:* Expand and prune, favoring coherent paths like "The woman has a dog in the park."
- *Result:* More structured than greedy.

Random Sampling

Random Sampling

- *Description: Sample the next token from the full multinomial distribution (proportional to probabilities).*
- *Pros: Highly stochastic, promotes diversity.*
- *Cons: Can produce incoherent or nonsensical text due to low-probability choices.*

Random (GPT-2)

Output: "It was a dark and stormy night. It alarmed me with its sudden outburst of..." (jumps erratically).

Top-k Sampling

Top-k Sampling

- *Description: Restrict sampling to the top-k most probable tokens (e.g., $k=50$), renormalizing their probabilities to sum to 1.*
- *Pros: Adaptive filtering of outliers while allowing variety.*
- *Cons: Fixed k may not suit all distributions (too restrictive for low-entropy, too loose for high).*

Top-k ($k=50$, GPT-2 on prompt)

Output: "You will think, Doctor, that I am mad..." (coherent with some creative deviation).

Top-p (Nucleus) Sampling

Top-p (Nucleus) Sampling

- *Description: Sample from the smallest set of tokens whose cumulative probability mass exceeds p (e.g., $p=0.9$), renormalizing within that nucleus.*
- *Pros: Dynamically adapts—more options in uncertain (high-entropy) contexts, fewer in confident ones.*
- *Cons: Still stochastic, may occasionally stray.*

Top-p ($p=0.9$, GPT-2)

Output: "Mr. Holmes had been talking to me about the case when..." (creative yet grounded).

Temperature Sampling

Temperature Sampling

- *Description: Adjust the logits before softmax with a temperature T :*

$$y_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T=1$: Standard softmax.
- $T>1$: Flattens the distribution (increases entropy, more random).
- $T<1$: Sharpens it (decreases entropy, more peaky like greedy).
- As $T \rightarrow 0$: (one-hot).

Often combined with top-k or top-p.

Temperature Examples (GPT-2, varying T on same prompt)

- $T=0.1$: "I am sorry, Mr. Holmes, but I cannot help you." (reitive, safe).
- $T=0.5$: "The inspector nodded slowly, his eyes narrowing." (coherent, varied).
- $T=1.1$: "Suddenly, a shadow moved in the corner of the room—or was it?" (diverse, slightly odd).
- $T=1.5$: "Zog flibberty gibbet in the quantum flux!" (incoherent gibberish).

References

- [Machine Learning](#)
- [Neural Networks](#)
- [Linear Algebra](#)
- [Attention is All You Need](#)
- [BERT](#)
- [GPT](#)
- [T5](#)